

Proxmox やりましょう

terudoru

目次

1 Proxmox VE とは	1
2 この記事を書いた理由	2
3 Proxmox VE のインストール	2
3.1 インストール前に用意するもの	2
3.2 インストールする	2
3.3 初期セットアップをする	4
3.4 おすすめの使い方	4
4 自宅鯖弄り録	5
4.1 用途ごとに VLAN で区切る	6
4.2 クラスタを作る	6
4.3 管理スイッチを設定する	7
4.4 VM/LXC 用 bridge を分け、サービスを VLAN 20 へ移す	8
4.5 home.arpa で名前解決する	9
4.6 Codex から Proxmox を操作する (ProxmoxMCP-Plus)	10
4.7 Zabbix で監視する	15
4.8 余談 : IOMMU と PCI passthrough	15
5 おわりに	15
参考にしたサイト	16

1 Proxmox VE とは

- ProxmoxVE は Debian ベースの仮想化基盤 [1]
- WebGUI での直感的な操作が特徴的で、数回ボタンをクリックするだけですぐ VM,LXC コンテナを建てることができる
- 思う存分サービスを破壊できる
- 無料で使える (嬉)

※ VM: 仮想マシン。LXC: 軽量の Linux コンテナ。VLAN: 1 本の物理ネットワークを用途別に論理分割する仕組み。

2 この記事を書いた理由

自宅鯖/Proxmox を布教したい（盆栽的な楽しさがあります）。最近、自宅鯖をいじっていたときにいろいろあってノードごと破壊したので、組み直すついでにインストールから運用までをまとめました。

3 Proxmox VE のインストール

3.1 インストール前に用意するもの

- **物理マシン** : x86-64 マシンなら基本何でも良い。ただし、VM を動かすには仮想化支援機能（Intel VT-x / AMD-V）が有効になっている必要があります。最近では企業から払い下げられたシンククライアント（富士通の FUTRO や DELL の Wyse など）を買ってきてクラスタを組む人をよく見ます（kawaii）。
- **起動用 USB メモリ** : リンクから ISO を落とし（2026 年時点では V9.2 が最新）、Balena Etcherなどで USB メモリに書き込む。
- **NIC** : 基本はオンボードで十分。10G 回線と 10G スイッチがあるなら、X520-DA2 などの 10G NIC（図 1）を足すと高速になって良い。
- **管理用 PC** : Proxmox の Web UI に入るための PC。ブラウザが動けば何でも良い。
- **モニター** : セットアップ時に使用。



<https://www.proxmox.com/en/downloads/proxmox-virtual-environment/iso>

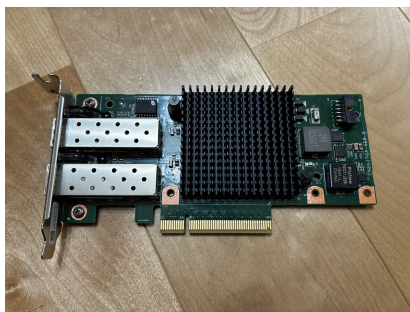


図 1 X520-DA2。Aliexpress で 5 千円ぐらいで手に入ります。

3.2 インストールする

Proxmox VE の ISO を USB メモリへ書き込み、USB から起動します。

起動時に各機種のブートメニューを開き、Proxmox VE を書き込んだ USB メモリを選びます。



図 2 Proxmox VE 9.2 のインストールメニュー。この画面まで来たら Install Proxmox VE (Graphical) を選びます。

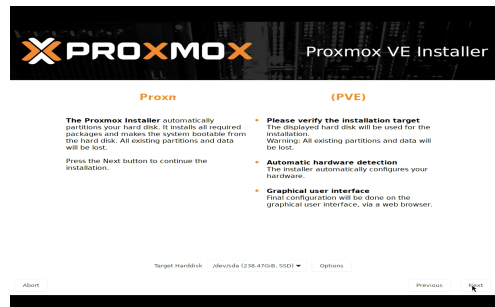


図 3 インストール先ディスクの選択。対象ディスクは初期化されます。

画面に従って地域、タイムゾーン、キーボード配列、root パスワード、通知用メールアドレスを設定します。

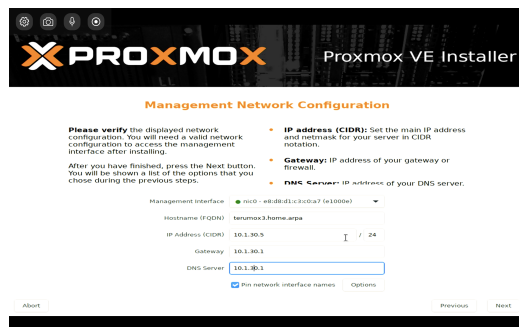


図 4 管理ネットワーク設定。Proxmox の Web UI にアクセスするための IP アドレスを、環境に合わせて設定してください。このあとの Summary 画面で設定を確認して Install を押します。

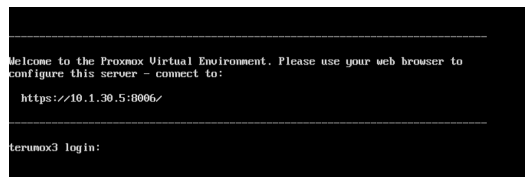


図 5 インストール完了後のコンソール。https://先ほど設定した IPv4 アドレス:8006/ が表示されれば、管理 PC から Proxmox VE の Web UI へ入る準備ができています。

インストール後は、管理 PC のブラウザから表示された URL にアクセスします。root ユーザーでログインして、次のような Web UI が表示されればインストールは成功です。

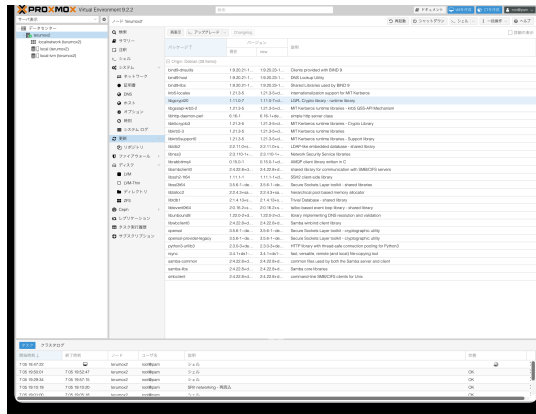


図 6 ブラウザから Proxmox VE の Web UI へ入った状態。

3.3 初期セットアップをする

まず、サブスクリプションなしでもアップデートが止まらないよう、Proxmox VE Helper-Scripts の PVE Post Install でリポジトリを整えます [2]。Enterprise リポジトリの無効化、No-Subscription リポジトリの有効化、subscription nag の抑制、Proxmox VE の更新までを対話式でまとめて済ませられます。

```
# ProxmoxホストのShellで初期セットアップを対話実行する。
bash -c "$(curl -fsSL https://raw.githubusercontent.com/community-scripts/ProxmoxVE/main/tools/pve/post-pve-install.sh)"
```

PVE Post Install

<https://community-scripts.org/scripts/post-pve-install>



質問には次の方針で答えます。

1. Start the Proxmox VE Post Install Script (y/n)? で y を選ぶ。
2. pve-enterprise と ceph enterprise は disable、pve-no-subscription は yes、pve-test は不要なら no を選ぶ。
3. subscription nag は yes で無効化し、Disable high availability? は no を選ぶ。
4. Proxmox VE の更新と再起動は、どちらも yes を選ぶ。

3.4 おすすめの使い方

Proxmox VE Helper-Scripts は、初期セットアップだけでなく、VM/LXC を作る時にも便利です [2]。おすすめを 6 つに絞ります。

Proxmox VE Helper-Scripts

<https://community-scripts.github.io/ProxmoxVE/>



Docker VM

Docker を VM で動かす。LXC 派閥もいて、どちらが良いのやら…

Home Assistant OS

家電、センサー、スマートプラグなどをまとめる。

AdGuard Home

家庭内 DNS と広告ブロックに使う。設定は katori_m 氏の記事を参照 [3]。

Tailscale LXC

外出先からのリモートアクセスに使う。

Cloudflared

ポート開放せずサービスを公開する。我が家では Minecraft サーバーに使用。

Zabbix

サーバーやネットワークを監視し、見ながらニャニャする。

※ Docker: アプリをコンテナとして動かす仕組み。

※ DNS: 名前から IP アドレスを引く仕組み。

※ Cloudflare Tunnel: 自宅側から Cloudflare へ接続してサービスを公開する仕組み。

4 自宅鯖弄り録

Proxmox のユースケースとして、我が家の環境を組み直したときの作業をまとめました。
役に立つかは謎です。

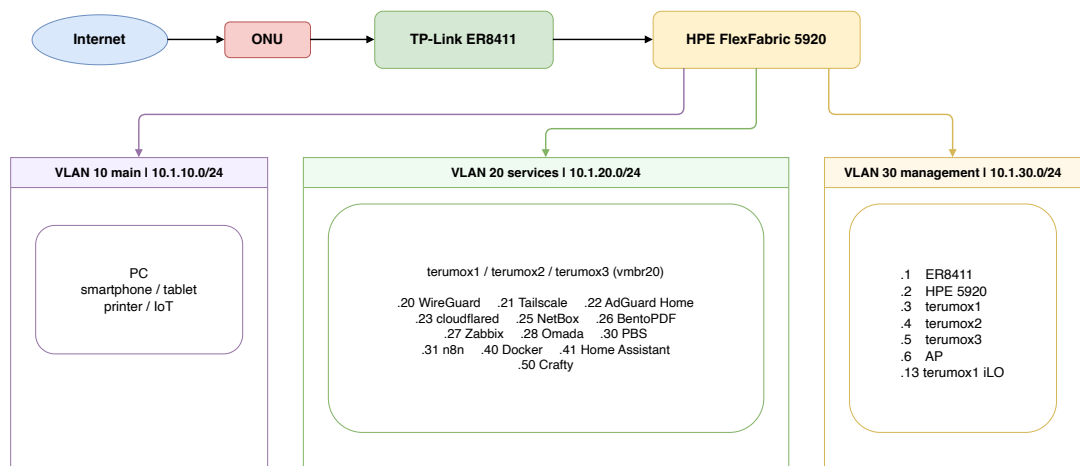


図 7 我が家の構成



図 8 我が家のサーバー群。撮影後にコアスイッチの電源が溶けたので入れ替えました。



図 9 新コアスイッチ、HPE FlexFabric 5920 (SFP+×24)。うるさすぎる。

4.1 用途ごとに VLAN で区切る

役割ごとに VLAN で区切って運用しやすいようにします。普段使いは VLAN 10、サービスは VLAN 20、インフラ管理は VLAN 30 に分けました。

VLAN 10 (main)	10.1.10.0/24、gateway 10.1.10.1
VLAN 20 (services)	10.1.20.0/24、gateway 10.1.20.1
VLAN 30 (management)	10.1.30.0/24、gateway 10.1.30.1
管理スイッチ	10.1.30.2
terumox1 Proxmox	10.1.30.3
terumox2 Proxmox	10.1.30.4
terumox3 Proxmox	10.1.30.5

内部 DNS のドメイン※は、Bonjour/mDNS※の .local と衝突しないよう home.arpa にします。ha.home.arpa、crafty.home.arpa、mc.home.arpa のようになります。

※ ドメイン: 名前解決で使う名前のまとまり。

※ Bonjour/mDNS: 同一 LAN 内で機器名を自動発見する仕組み。

4.2 クラスタを作る

quorum (過半数のノードが稼働している状態) を確保するため 3 台構成にしています。2 台だと 1 台落ちた時点で過半数を割りますが、3 台なら 1 台落ちても残り 2 台で保てます。

まず名前解決を揃えます。画面に出てくる Link 0 は、Corosync (クラスタ管理の仕組み) がノード間通信に使う経路です。ノード名と IP の対応がずれていると参加に失敗するため、先に 3 台すべての /etc/hosts へ同じ 3 行を入れて固定しました。

```
1 | # 3台すべての /etc/hosts に入れ、ノード名を管理VLANのIPへ固定する。
2 | 10.1.30.3 terumox1.home.arpa terumox1
3 | 10.1.30.4 terumox2.home.arpa terumox2
```

なお、Proxmox の公式ドキュメントでは Corosync の link address に IP アドレスを直接使う方が安全とされています。Web UI で明示的に IP を選べる場合は選んでおきましょう。

名前解決が揃ったら、1 台目でクラスタを作成します。terumox1 の Web UI で「データセンター」→「クラスタ」を開き、「クラスタを作成」を押します。クラスタ名に terumox-cluster を入れ、Link 0 には管理 VLAN の IP 10.1.30.3 を選んで「作成」を押します。

作成できたら、同じ画面の「Join 情報」を開き、「情報をコピー」で丸ごとコピーしておきます。

続いて terumox2、terumox3 を順番に参加させます。ノードの Web UI で「クラスタ」→「クラスタに参加」を開き、Join 情報を「情報」欄へ貼り付けると Peer アドレスと指紋が自動で埋まります。あとは terumox1 の root パスワードを入力し、Link 0 にそのノード自身の管理 VLAN IP (terumox2 なら 10.1.30.4) を選んで参加します。

実際の作業では、terumox3 の参加が途中で失敗しました。pvecm status で半端な登録を確認し、失敗ノードを削除して参加をやり直すことで復旧しました。削除の手順は既存の実例を参考にしました [6]。

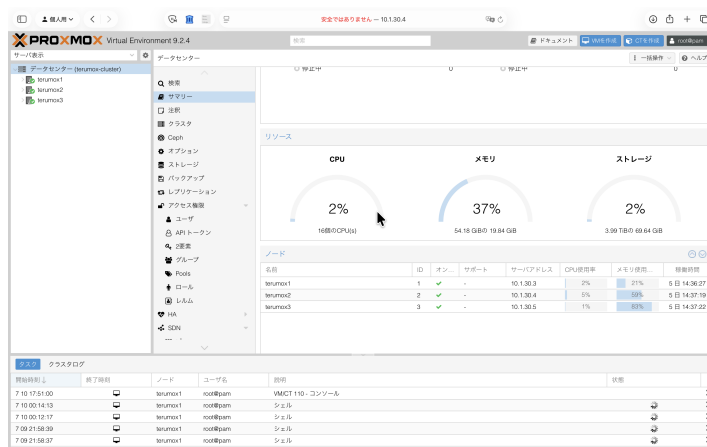


図 10 クラスタ作成後のデータセンター画面。terumox1～terumox3 の 3 ノードが terumox-cluster として 1 つにまとまっています。

4.3 管理スイッチを設定する

スイッチ側では、先に決めた VLAN を作成し、接続先に応じて各ポートの役割を設定します。設定画面やコマンドは機種ごとに異なるため、ここでは設定の内容だけ大まかに書きます。収容例を図 11 に示します。

※ trunk: 複数 VLAN をまとめて通すポート。

※ access: 1 つの VLAN だけに所属するポート。
※ native VLAN: trunk でタグなし通信として扱う VLAN。
※ tagged: VLAN tag 付き通信として扱う状態。

- 複数 VLAN を扱う機器へのポートは trunk にして必要な VLAN を tagged で通し、単一用途の機器へのポートは access にして所属する VLAN を 1 つ指定する。
- native VLAN は接続する両側で揃え、管理 IP を管理用 VLAN へ移す場合は、変更後も管理画面へ到達できる経路を先に確保する。
- 設定後は、各ポートの VLAN への所属、trunk で許可した VLAN、管理 IP への到達性を確認し、再起動後も残るよう設定を保存する。

4.4 VM/LXC 用 bridge を分け、サービスを VLAN 20 へ移す

Proxmox では、管理用 bridge と VM/LXC 用 bridge を分けます。オンボード NIC の nic0 は管理用 bridge vmbr0 に入れ、各ノードを設計表のとおり 10.1.30.3~10.1.30.5 の VLAN 30 に置きます。X520-DA2 の 1 ポート目 nic1 は、新しく作る Linux bridge vmbr20 に入れます。

「System」→「Network」→「Create」→「Linux Bridge」で、Name に vmbr20、Bridge ports に nic1 を入れます。IPv4/CIDR、Gateway、IPv6/CIDR は空欄、Autostart は有効、VLAN aware は無効にします。HPE 5920 側の接続ポートを VLAN 20 の access port にしているため、VLAN tag は Proxmox 側で付けず、物理スイッチ側で収容します。

管理スイッチでのVLAN収容例（機種非依存）

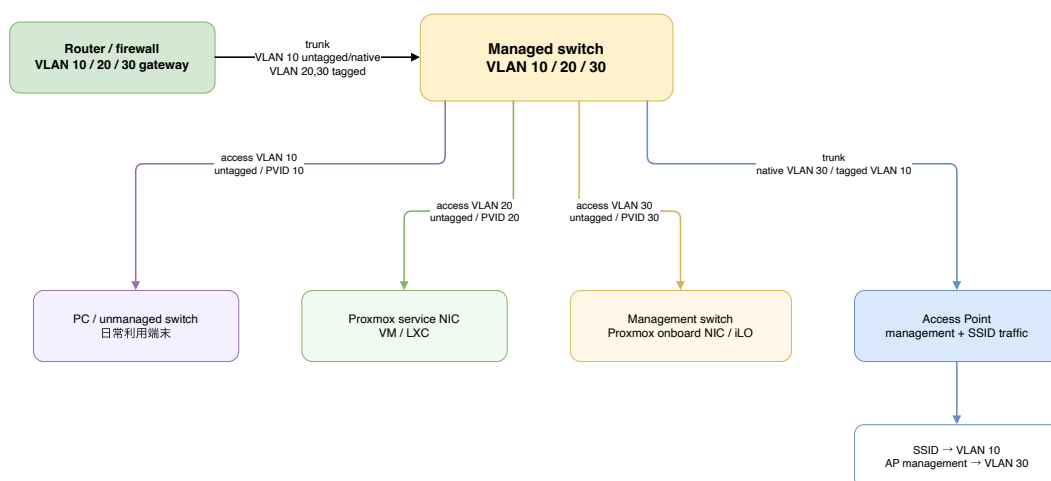


図 11 管理スイッチにおける VLAN の収容例。接続先に応じて trunk または access を選び、接続する両側で VLAN 設定を一致させます。

図 12 terumox3 で vbr20 を作成する画面。この bridge は VM/LXC 用の VLAN 20 通信だけを受け持ちます。

保存して反映後、vbr0 が管理用 IP を持ったまま残り、vbr20 が nic1 をポートとして持っていれば意図通りです。これで、管理経路はオンボード NIC 側、VM/LXC のサービス通信は X520-DA2 側に分かります。

続いて、サービス用の VM/LXC の仮想 NIC を vbr20 へ移し（VLAN Tag は空欄）、VLAN 20 の固定 IP を割り当てました。たとえば、AdGuard Home は 10.1.20.22、Zabbix は 10.1.20.27、Crafty Controller は 10.1.20.50 です。ほかのサービスも同じ帯域で重複しないように管理しています。

変更後は、クラスタが quorum を保っていること、3 ノードが online のままであること、各 VM/LXC が vbr20 を使っていることを確認します。

4.5 home.arpa で名前解決する

わかりやすい名前でアクセスできるよう、AdGuard Home に DNS rewrite を登録しました。

```

1 # DNS rewriteの例。ポート番号はDNS名ではなくURL側に付ける。
2 terumox1.home.arpa -> 10.1.30.3
3 terumox2.home.arpa -> 10.1.30.4
4 terumox3.home.arpa -> 10.1.30.5
5 adguard.home.arpa -> 10.1.20.22
6 crafty.home.arpa -> 10.1.20.50

```

設定後は `dig @10.1.20.22 terumox1.home.arpa +short` のように AdGuard へ直接問い合わせを確認します。あとは、ER8411 の DHCP 設定で VLAN 10/20/30 の配布 DNS を 10.1.20.22 にすれば、通常の端末も AdGuard を見るようになります。

自宅 LAN の外から Tailscale 経由でアクセスする場合は、home.arpa の名前解決を 10.1.20.22 へ転送する split DNS を Tailscale 側に設定します。

※ DNS rewrite: 特定の名前を指定した IP へ返す DNS 側の固定登録。

※ split DNS: 特定ドメインだけ別の DNS サーバーへ問い合わせる設定。

4.6 Codex から Proxmox を操作する (ProxmoxMCP-Plus)

最近流行りのコーディングエージェントを用いて Proxmox を操作できたら面白そうだなと思って試しました。ProxmoxMCP-Plus[7] という OSS を使います。

ProxmoxMCP-Plus

<https://github.com/RekklesNA/ProxmoxMCP-Plus>



Codex へ root パスワードを渡すのは怖いので、Proxmox 側に専用ユーザーと API token を作ります。普段使う token は読み取りと低リスク操作に絞り、Administrator token は別にし必要なときだけ使うことにしました。token なら用途ごとに権限を分けられ、不要になったときはパスワードを変更せずに失効できます。

※ API token: パスワードの代わりにアプリへ渡す認証用の鍵。

※ role: Proxmox で複数の権限をまとめたもの。

※ ACL: どのユーザーが、どの範囲で、どの role を使えるかを定める設定。

常用ユーザーと API token を作る

Proxmox の Web UI で「Datacenter」→「Permissions」→「Users」→「Add」を開き、codex@pveを作ります。続いて「API Tokens」→「Add」で、同じユーザーに codex という token を作ります。Privilege Separation は有効にします。表示される token secret は一度しか確認できないため、その場でコピーして安全な場所へ保存します。

CLI で作る場合は、Proxmox ホストの Shell で次のように実行できます。token 作成時に表示された値は原稿や Git へ書きません。

```
1 # Codex専用ユーザーと、権限分離した読み取り用tokenを作る。
2 pveum user add codex@pve --comment "Codex MCP automation user"
3 pveum acl modify / -user codex@pve -role PVEAuditor
4 pveum user token add codex@pve codex -privsep 1 \
5   --comment "Codex MCP read-only token"
6 pveum acl modify / -token 'codex@pve!codex' -role PVEAuditor
7
8 # ユーザー本体とtokenに付いた権限を確認する。
9 pveum user permissions codex@pve
10 pveum user token permissions codex@pve codex
```

ここで注意するのが Privilege Separation です。有効にした token では、元ユーザーと token の両方へ権限を付ける必要があります。最初は token 側だけに PVEAuditor を付けましたが、Codex から見えたのはノード一覧だけで、ストレージや VM/LXC の一覧は空でした。最終的に、Datacenter の Permissions へ次の 2 つが入った状態で取得できました。

```
1 # Pathは /、Propagateは有効にする。
```

```
2 / codex@pve          PVEAuditor  true
3 / codex@pve!codex    PVEAuditor  true
```

ProxmoxMCP-Plus の接続設定を作る

ProxmoxMCP-Plus の example を config.json としてコピーし、接続先、ユーザー、token 名、token secret を書き換えます。我が家の最初の接続先は terumox1 の管理 IP 10.1.30.3:8006 です。

```
1 # exampleをコピーし、ローカル専用の設定ファイルを作る。
2 cp proxmox-config/config.example.json \
3   proxmox-config/config.json
4 nano proxmox-config/config.json
```

設定する主な項目は次のとおりです。token_value には、先ほどコピーした secret を入れます。

```
1 "proxmox": {
2   "host": "10.1.30.3",
3   "port": 8006,
4   "verify_ssl": false,
5   "service": "PVE"
6 },
7 "auth": {
8   "user": "codex@pve",
9   "token_name": "codex",
10  "token_value": "ここにtoken secret"
11 },
12 "security": {
13   "dev_mode": true
14 }
```

初期状態の Proxmox は自己署名証明書を使うため、最初の疎通確認では verify_ssl=false にしました。ProxmoxMCP-Plus では、この設定を使う場合に security.dev_mode=true も必要です。長期運用では証明書を信頼できる状態にして、verify_ssl=true と dev_mode=false へ戻します。

secret 入りファイルは自分だけが読めるようにし、Git へ追加しません。

```
1 # token secretを含む設定ファイルの権限を制限する。
2 chmod 600 proxmox-config/config.json
```

```
3 | git status --short proxmox-config/config.json
```

Codex へ常用 MCP を登録する

Codex の config.toml には、ProxmoxMCP-Plus を uvx で起動する MCP サーバーを登録します。読み取り系だけを自動承認し、起動、停止、snapshot 作成などの状態変更は毎回確認する設定にしました。削除、rollback、restore、SSH 経由の command execution は常用側へ公開しません。

```
1 | # ~/.codex/config.toml に追記する。
2 | # 同名の見出しがある場合は重複させず、中身を置き換える。
3 | [mcp_servers.proxmox]
4 | command = "uvx"
5 | args = ["proxmox-mcp-plus"]
6 | env = { PROXMOX_MCP_CONFIG = "/path/to/proxmox-config/config.json" }
7 | default_tools_approval_mode = "prompt"
8 | enabled_tools = [
9 |     "get_nodes",
10 |    "get_cluster_status",
11 |    "get_storage",
12 |    "get_vms",
13 |    "get_containers",
14 |    "list_snapshots",
15 |    "start_vm",
16 |    "start_container",
17 |    "stop_container",
18 |    "create_snapshot",
19 |    "list_jobs",
20 |    "get_job",
21 |    "poll_job"
22 | ]
23 |
24 | # 一覧取得などの読み取り系だけを自動承認する。
25 | [mcp_servers.proxmox.tools.get_nodes]
26 | approval_mode = "approve"
27 |
28 | [mcp_servers.proxmox.tools.get_cluster_status]
29 | approval_mode = "approve"
30 |
```

```

31 | [mcp_servers.proxmox.tools.get_storage]
32 | approval_mode = "approve"
33 |
34 | [mcp_servers.proxmox.tools.get_vms]
35 | approval_mode = "approve"
36 |
37 | [mcp_servers.proxmox.tools.get_containers]
38 | approval_mode = "approve"
39 |
40 | [mcp_servers.proxmox.tools.list_snapshots]
41 | approval_mode = "approve"

```

stop_vm は ProxmoxMCP-Plus の tool 定義上 force stop になるため、常用側の enabled_tools から外しました。VM を停止するときは、まず guest OS を正常終了させる graceful shutdown を使います。

読み取りだけで接続を確認する

設定を反映するため Codex を再起動し、最初は次のように変更を禁止したうえで一覧取得だけを依頼しました。

```

1 | # Codexへの最初の依頼。
2 | Proxmoxのnode、storage、VM、LXC一覧を取得して。
3 | 変更操作はしないで。

```

権限を修正したあとの取得結果は次のとおりです。3 ノード、2 つの storage、VM 1 台、LXC 13 台を読み取れました。

```

1 | nodes:
2 | terumox1 online
3 | terumox2 online
4 | terumox3 online
5 |
6 | storage:
7 | local-lvm
8 | local
9 |
10 | VM:
11 | 100 docker
12 |

```

```
13 | LXC:
14 | 101 adguard
15 | 102 cloudflared
16 | 103 wireguard
17 | 104 tailscale
18 | 105 homeassistant
19 | 106 crafty-controller
20 | 107 proxmox-backup-server
21 | 108 ubuntu
22 | 109 n8n
23 | 110 zabbix
24 | 111 upsnap
25 | 112 netbox
26 | 113 bentopdf
```

ここまで読めることを確認してから、snapshot 作成、VM/LXC の起動、LXC の graceful shutdown、job 確認を追加しました。変更操作を頼むときは、先に node、VMID/CTID、名前、現在状態を取得し、対象を確認してから実行します。

管理作業用 token を分離する

すべての設定作業を Codex 経由で行う場合でも、常用 token を Administrator へ昇格させません。codex-admin@pve と codex-admin@pve!admin を別に作り、ユーザー本体と token の両方へ Administrator を付けます。

```
1 | # DatacenterのPermissionsで確認する管理用ACL。
2 | / codex-admin@pve Administrator true
3 | / codex-admin@pve!admin Administrator true
```

管理用の secret は config.admin.json へ保存し、常用設定と混ざらないようにします。Codex 側でも proxmox_admin という別名で登録し、読み取りを含むすべての tool を実行前確認にしました。

```
1 | # ~/.codex/config.toml に追加する管理作業用MCP。
2 | [mcp_servers.proxmox_admin]
3 | command = "uvx"
4 | args = ["proxmox-mcp-plus"]
5 | env = { PROXMOX_MCP_CONFIG = "/path/to/proxmox-config/config.admin.json" }
6 | default_tools_approval_mode = "prompt"
```

管理用 namespace では、削除、rollback、restore、ISO や backup の削除、command execution、VM の force stop まで見えます。使う前に対象、目的、戻し方を確認し、作業後は MCP 設定または token を無効にします。実際の確認では、terumox-cluster が quorum を保ち、3 ノードが online であることまで読み取れました。

設定ファイル一式と、token secret を履歴に残さず保存する補助スクリプトは、原稿付属の補足資料にも載せています。

ProxmoxMCP-Plus 完全設定メモ

<https://github.com/terudoru/edp2026-article/blob/main/supplements/proxmox-mcp-plus-setup.md>



4.7 Zabbix で監視する

サーバーが増えてくると監視が欲しくなるので、Zabbix を導入しました。

まずはミライサーバーの記事を見ながら、Zabbix 本体のインストール、DB の作成などを行うとよいです [4]。

Zabbix 本体の導入手順

https://www.miraiserver.ne.jp/column/about_zabbix-install-ubuntu/



日本語を選択できない場合は、language-pack-ja を導入して ja_JP.UTF-8 を生成し、システムの言語設定へ反映します。

Zabbix が立ち上がったら、次は Proxmox を監視対象として追加します。Proxmox VE は API を備えており、Zabbix 側にも Proxmox VE by HTTP テンプレートがあります。IT ログの記事では、Proxmox 側で監視用ユーザーを作り、API token を発行し、Zabbix 側でホスト追加とマクロ設定を行う流れが画面付きでまとまっています [5]。

Proxmox VE を Zabbix で監視する手順

<https://www2.filewo.net/wordpress/2025/05/19/4294/>



※ API token: パスワードの代わりにアプリへ渡す認証用の鍵。

※ template: Zabbix で監視項目や取得方法をまとめたひな形。

※ SNMP: スイッチやルーターから状態を読むためによく使われる監視用プロトコル。

4.8 余談 : IOMMU と PCI passthrough

ほかにも、ルーター VM を試す過程で IOMMU や PCI passthrough を触りました。このへんは本筋から外れるため、設定手順、確認コマンド、検証結果を GitHub に載せておきます。

Proxmox IOMMU 確認・設定メモ

<https://github.com/terudoru/edp2026-article/blob/main/supplements/iommu-notes.md>



5 おわりに

このままだと締まりが悪いので、好きなコンテンツの二次元コードを貼って終わりにしたいと思います。



参考にしたサイト

- [1] Proxmox Server Solutions GmbH, “Proxmox Virtual Environment - Open-Source Server Virtualization Platform”,

<https://www.proxmox.com/en/products/proxmox-virtual-environment/overview>



- [2] Community Scripts, “Proxmox VE Helper-Scripts”,

<https://community-scripts.github.io/ProxmoxVE/>



- [3] katori_m, “AdGuard Home で広告ブロック&お手軽内部ネットワーク用 DNS を作ろう (ついでに冗長化)”, Qiita,

https://qiita.com/katori_m/items/02aef27024b848b160a4



- [4] ミライサーバーのススメ, “はじめての Zabbix 構築 Ubuntu24.04/22.04 で監視サーバーを立ち上げる実践ガイド”,

https://www.miraiserver.ne.jp/column/about_zabbix-install-ubuntu/



- [5] tukapiyo, “Proxmox VE を Zabbix で監視する”, IT ログ,

<https://www2.filewo.net/wordpress/2025/05/19/4294/>



- [6] wildrick, “Proxmox クラスタからノードを削除する手順”,

<https://note.com/wildrick/n/nbcfdbb8dffa4>



- [7] RekklesNA, “ProxmoxMCP-Plus”,

<https://github.com/RekklesNA/ProxmoxMCP-Plus>



- [8] w453y, “[Guide] How I turned a Proxmox cluster node into standalone (without reinstalling it)”, Reddit,

https://www.reddit.com/r/Proxmox/comments/1k4slor/guide_how_i_turned_a_proxmox_cluster_node_into/



- [9] karlsiu1, “Joining Cluster pve-ssl.pem error”, Proxmox Support Forum,

<https://forum.proxmox.com/threads/joining-cluster-pve-ssl-pem-error.131263/>



- [10] Important_Reserve460, “[Guide] How to enable IOMMU for PCI Passthrough”, Reddit,

https://www.reddit.com/r/Proxmox/comments/1brl7vw/guide_how_to_enable_iommu_for_pci_passthrough/?tl=ja



- [11] zayigo, “Intel X520-DA2 IOMMU groups”, Proxmox Support Forum,

<https://forum.proxmox.com/threads/intel-x520-da2-iommu-groups.118261/>



- [12] TenForward, “SR-IOV を有効にする”,

<https://tenforward.hatenablog.com/entry/20111206/1323171401>



- [13] えのきこ, “Proxmox で Intel NIC を SR-IOV”, note,

<https://note.com/enokiko/n/n4b4b4e4c399b>

